SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)
A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.
Fix issues related to bitwise operations and endianness handling.
Optimize the implementation to improve processing speed for large files (≥50 MB).
Evaluate how secure hash functions improve integrity compared to basic checksum methods.

---

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation
A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption).
Analyze the performance vs security trade-offs in TLS optimization.

---

3. Debugging and Optimizing Digital Signature Algorithm (DSA)
A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.
Fix errors in signature generation and verification steps.
Optimize modular arithmetic operations for faster execution.
Evaluate the impact of randomness quality on signature security.

---

4. Debugging Key Management System
A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).
Fix issues related to key lifecycle management (generation, distribution, rotation).
Optimize secure storage using encryption and access control mechanisms.
Analyze how poor key management can compromise an entire cryptographic system.

---

5. Optimization of Secure File Transfer Protocol (SFTP-like System)
A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization.
Fix errors in packet handling and integrity verification.
Optimize throughput for large file transfers (≥100 MB).
Compare the optimized system with insecure file transfer methods and justify the improvements.

---

1. Debugging Hash Function Implementation (SHA Family)

A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

A. Identify errors in message padding and block processing steps.

The SHA algorithm requires proper padding and block division before hashing. Common implementation errors include:

- Incorrect calculation of padding bits.
- Missing length field at the end of the message.
- Improper division of data into 512-bit blocks.
- Processing blocks in the wrong order.
- Buffer overflow due to incorrect indexing.

If the padding process is incorrect, the same input may produce different hash values.

B. Fix issues related to bitwise operations and endianness handling.

SHA algorithms rely heavily on bitwise operations such as:

- AND (&)
- OR (|)
- XOR (^)
- NOT (~)
- Left shift (<<)
- Right shift (>>)

- Circular rotation

Common issues include:

- Using arithmetic shift instead of logical shift.
- Incorrect implementation of circular rotation.
- Confusion between little-endian and big-endian byte order.
- Improper conversion between bytes and integers.

Fixes:

- Use circular rotation functions.
- Ensure all blocks follow big-endian representation.
- Apply proper masking using `0xFFFFFFFF`.
- Validate bitwise operations using test vectors.

C. Optimize the implementation for large files (≥ 50 MB).

Optimization techniques include:

- Read files in chunks instead of loading the entire file into memory.
- Use buffered I/O operations.
- Reduce unnecessary memory allocation.
- Process blocks in parallel where possible.
- Use optimized libraries such as `hashlib`.

Optimized techniques:

- Chunk processing reduces memory usage.
- Buffering improves throughput.
- Native implementations provide better performance.

D. Evaluate how secure hash functions improve integrity compared to basic checksum methods.

Secure Hash Functions

- Resistant to collisions.
- Detect small modifications.
- One-way transformation.
- Computationally secure.

Basic Checksums

- Easy to compute and modify.
- Vulnerable to collisions.
- Provide weak integrity guarantees.

Secure hash functions such as SHA-256 offer significantly stronger protection than simple checksum algorithms.

---

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

A Python-based SSL/TLS handshake simulation shows delays and handshake failures.

A. Debug issues in certificate validation and key exchange steps.

Common issues include:

- Expired certificates.
- Invalid certificate chains.
- Incorrect public keys.
- Mismatched encryption algorithms.
- Errors during Diffie-Hellman key exchange.

Solutions:

- Verify certificate signatures.
- Check certificate expiration dates.
- Validate the certificate authority.
- Ensure both parties support the same cipher suite.

B. Optimize the handshake process to reduce latency.

Handshake latency can be reduced by:

- Using session caching.
- Reducing certificate size.
- Selecting efficient cipher suites.
- Minimizing network round trips.
- Using TLS 1.3.

Benefits:

- Faster connection establishment.
- Reduced computational overhead.
- Improved user experience.

C. Refactor the code to reuse session keys efficiently (session resumption).

Session resumption allows clients to reuse previously established session parameters.

Methods include:

- Session IDs.
- Session tickets.
- Pre-shared keys.

Advantages:

- Lower latency.
- Reduced CPU usage.
- Faster reconnection.

D. Analyze performance versus security trade-offs.

Optimization	Performance Gain	Security Impact
Session resumption	High	Slight reduction
Smaller key size	High	Lower security
TLS 1.3	High	Strong security
Cipher simplification	Medium	Moderate risk

Security should never be sacrificed excessively for performance.

---

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

A Python implementation of DSA generates invalid signatures during verification.

A. Identify issues in parameter selection and random number generation.

Possible issues include:

- Invalid prime numbers.
- Weak parameter selection.
- Reuse of random values.
- Predictable random generators.
- Incorrect key sizes.

Poor randomness can compromise the entire signature scheme.

B. Fix errors in signature generation and verification steps.

Signature generation steps

1. Generate random value `k`.
2. Compute `r`.
3. Compute `s`.
4. Generate the signature pair `(r, s)`.

Verification steps

1. Compute the message hash.
2. Compute intermediate values.
3. Calculate verification values.
4. Compare with the received signature.

Corrections:

- Validate parameter ranges.
- Generate cryptographically secure random numbers.
- Verify modular arithmetic calculations.
- Check for division errors.

C. Optimize modular arithmetic operations.

Optimization methods:

- Use fast modular exponentiation.
- Use square-and-multiply algorithms.

- Cache repeated calculations.
- Use optimized libraries.

Benefits include:

- Reduced execution time.
- Faster verification.
- Lower computational cost.

D. Evaluate the impact of randomness quality on signature security.

Weak randomness can lead to:

- Private key leakage.
- Signature forgery.
- Replay attacks.
- Complete system compromise.

Cryptographically secure random number generators are essential for DSA security.

---

4. Debugging Key Management System

A Python-based key management system fails to securely store and retrieve cryptographic keys.

A. Identify flaws in key storage.

Common flaws include:

- Plaintext storage.
- Hard-coded keys.
- Unencrypted databases.
- Weak passwords.
- Lack of access control.

These vulnerabilities allow attackers to steal secret keys.

B. Fix issues related to key lifecycle management.

Key lifecycle stages:

1. Key generation.
2. Key distribution.
3. Key usage.
4. Key rotation.
5. Key revocation.
6. Key destruction.

Improvements:

- Generate keys securely.
- Rotate keys periodically.
- Revoke compromised keys.
- Destroy expired keys.

C. Optimize secure storage using encryption and access control.

Optimization methods:

- Encrypt keys using AES.
- Use hardware security modules (HSMs).
- Implement role-based access control.
- Store audit logs.
- Use multi-factor authentication.

Benefits:

- Stronger security.
- Better scalability.
- Reduced risk of unauthorized access.

D. Analyze how poor key management compromises cryptographic systems.

Even strong encryption algorithms become useless if keys are compromised.

Consequences include:

- Data breaches.
- Identity theft.
- Financial losses.
- Unauthorized access.
- Complete failure of security mechanisms.

Cryptography is only as strong as its key management system.

---

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

A. Debug issues in encryption and decryption synchronization.

Possible causes include:

- Mismatched encryption keys.
- Packet loss.
- Incorrect initialization vectors.
- Block alignment errors.
- Transmission delays.

Solutions:

- Synchronize encryption parameters.
- Verify keys before transfer.
- Handle retransmissions correctly.

B. Fix errors in packet handling and integrity verification.

Packet-related problems:

- Missing packets.
- Duplicate packets.
- Corrupted packets.
- Sequence errors.

Solutions:

- Add sequence numbers.

- Use acknowledgments.
- Verify hashes.
- Detect packet loss.

C. Optimize throughput for large file transfers (≥ 100 MB).

Optimization techniques:

- Compress data before encryption.
- Increase packet size.
- Use multithreading.
- Implement buffering.
- Transfer files in parallel chunks.

Benefits:

- Reduced transfer time.
- Better bandwidth utilization.
- Lower processing overhead.

D. Compare the optimized system with insecure file transfer methods.

Feature	Insecure Transfer	Optimized Secure Transfer
Encryption	No	Yes
Integrity checking	No	Yes
Authentication	No	Yes
Packet verification	No	Yes
Data confidentiality	Low	High

Justification

The optimized secure transfer system ensures:

- Confidentiality through encryption.
- Integrity through hashing.
- Authentication through secure keys.
- Reliability through packet verification.

Although security mechanisms introduce additional computational overhead, they provide protection against interception, tampering, and unauthorized access.

Conclusion

Debugging and optimization are essential in cryptographic systems because implementation errors can compromise security even when strong algorithms are used. Proper debugging techniques, efficient optimization strategies, secure key management, and robust integrity verification ensure that cryptographic applications remain secure, reliable, and scalable.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation